

The Cartographer

LLM-Driven Pattern Catalog Expansion for Deterministic Synthesis Systems

David Jean Charlot, PhD Open Interface Engineering, Inc. (openIE)
University of California, Santa Barbara (UCSB) david@openie.dev |
dcharlot@ucsb.edu

February 2026 | Version 1.0

This work is licensed under CC BY-ND 4.0 | Open Access Research

Abstract

Large language models contain implicit knowledge of every computational pattern ever published — algorithms, data structures, system primitives, mathematical transforms. Yet this knowledge is trapped behind probabilistic text generation: to extract a sorting algorithm, one must prompt the model, wait seconds, parse its output, verify correctness, and hope for consistency. The knowledge exists but is inaccessible at machine speed.

We present the **Cartographer**, a system that uses LLMs not to generate code but to enumerate and type-sign computational patterns — extracting the catalog of all known computing from implicit model knowledge into an explicit, searchable, typed registry. Enumeration is cheap: the Cartographer extracts only typed signatures (name, input types, output types, keywords, description), deferring code generation to lazy synthesis on first use. A single enumeration pass across 48 domains produces approximately 4,800 typed pattern signatures at the cost of 48 LLM calls.

The resulting **DynamicCatalog** is a runtime-extensible, bincode-serialized registry with three index structures (name, keyword, domain) supporting exact lookup, Jaccard similarity search, and domain-scoped queries. Patterns are automatically deduplicated against both the static catalog (800 patterns) and previously discovered dynamic entries. The `dyn:` prefix namespace prevents collision with established pattern identifiers.

Integration with the Pattern programming language [1] is seamless: dynamic patterns participate in graph validation, composition table construction, and the full OODA engagement loop. Code generation occurs lazily — L3 (hybrid LLM synthesis) generates on first invocation, Lo (learned memory) caches the result permanently. Second invocation: microseconds.

Keywords: pattern enumeration, catalog expansion, LLM knowledge extraction, type inference, computational pattern database, lazy code generation, deterministic synthesis

1. Introduction

1.1 The Implicit Knowledge Problem

Every LLM trained on code contains knowledge of virtually every algorithm, data structure, and system primitive ever implemented. GPT-4, Claude, Llama — each can describe and implement a B-tree, a Kalman filter, a buddy allocator, a Viterbi decoder. This knowledge was absorbed during training, encoded across billions of parameters, and is accessible only through sequential text generation at human-readable speeds.

The AGI-Wisdom Code Factory [2] takes a fundamentally different approach: computational patterns are explicit, typed, pre-validated entries in a deterministic catalog. The system retrieves and composes patterns in microseconds — no generation, no probabilistic sampling, no temperature parameters. The static catalog contains 800 patterns across 28 domains, achieving 100% on HumanEval (164/164) [3] and MBPP (974/974) [4].

But 800 patterns, while sufficient for competitive benchmarks, do not cover all of computing. Operating system kernels, compiler internals, device drivers, window systems, build tools — these domains contain hundreds of additional computational patterns that the static catalog does not address. The question is not whether the LLM knows these patterns — it does — but how to extract that knowledge into the deterministic system where it can be retrieved at machine speed.

1.2 Enumeration Over Generation

The Cartographer answers this question with a key architectural insight: **enumerate, don't generate**. An LLM call that generates a complete implementation of a buddy allocator costs seconds and produces output of uncertain quality. An LLM call that produces a typed signature — `buddy_allocator : (Int, Int) -> Option[Int]` — costs the same time but produces output that is trivially parseable, type-checkable, and cacheable. The implementation can be generated later, on demand, by a separate subsystem (L3 hybrid synthesis) and cached permanently (LO learned memory).

This decomposition — signature enumeration by LLM, lazy implementation by L3, permanent caching by LO — converts the LLM from a runtime code generator into a one-time catalog builder. The LLM's contribution is bounded and monotonically decreasing: as the catalog grows, fewer patterns remain undiscovered. Eventually, enumeration completes. The LLM's job is finished. The deterministic system operates independently, forever.

1.3 Scope and Contribution

This paper describes:

1. **The enumeration protocol** — structured prompts that extract typed pattern signatures from LLMs across 48 domains of computing (Section 3)
2. **Robust type parsing** — a tolerant parser that maps LLM output (including aliases, alternate bracket styles, and unknown types) into the Pattern type system (Section 4)

3. **The DynamicCatalog** — a runtime-extensible, indexed, persistent pattern registry with Jaccard similarity search (Section 5)
 4. **Deduplication** — automatic filtering against both static (800 patterns) and previously discovered dynamic entries (Section 6)
 5. **Integration** — seamless participation in graph validation, composition tables, the OODA engagement loop, and lazy code generation (Section 7)
-

2. Architecture

2.1 Pipeline Overview

The Cartographer operates as a batch pipeline that converts implicit LLM knowledge into explicit catalog entries:

```
LLM (knows everything implicitly)
  → Cartographer (enumerate + type-sign + deduplicate)
    → DynamicCatalog (persistent runtime registry)
      → CodeFactory (lookup: Static 800 → Dynamic → L3)
        → L0 absorbs result (microsecond retrieval forever)
```

The pipeline is intentionally one-directional. The LLM is never consulted during runtime pattern resolution — only during catalog expansion. This architectural boundary ensures that the deterministic system’s microsecond latency is never degraded by LLM call overhead.

2.2 Domain Coverage

The Cartographer enumerates 48 domains — the 28 already covered by the static catalog’s DomainRecognizer plus 20 system-level domains:

Existing 28 domains: math, graph, dp, string, tree, search, sort, ml, distributed, database, networking, crypto, imaging, signal, control, quantum, physics, bioinformatics, verification, finance, optimization, compression, error_correction, graphics, robotics, ir, audio, gis

New 20 system-level domains: `os_kernel`, `filesystem`, `device_driver`, `window_system`, `shell`, `compiler_frontend`, `compiler_backend`, `virtual_machine`, `container`, `package_manager`, `build_system`, `text_editor`, `version_control`, `web_server`, `web_client`, `testing`, `logging`, `monitoring`, `serialization`, `concurrency`

At 100 patterns per domain, full enumeration targets approximately 4,800 patterns — a 6x expansion of the static catalog. In practice, many domains yield fewer than 100 distinct patterns, and deduplication removes cross-domain overlap. The expected catalog size after full enumeration is 3,000–4,000 unique typed signatures.

2.3 Component Architecture

The system comprises four primary components:

Component	Responsibility	Persistence
<code>Cartographer</code>	LLM interaction, prompt construction, response parsing, deduplication	Stateless (consumes provider)
<code>CartographerConfig</code>	Model, temperature, token limits, timeout, patterns-per-domain	In-memory
<code>DynamicCatalog</code>	Indexed storage, keyword search, domain queries, <code>PatternSig</code> conversion	<code>bincode (.bin)</code>
<code>DiscoveredPattern</code>	Individual pattern record with types, keywords, provenance	Serialized within catalog

3. Enumeration Protocol

3.1 Prompt Structure

Each domain enumeration uses a structured prompt that specifies the output format, type system, and constraints:

```

You are a computational pattern enumerator. Your task: list ALL known
computational patterns, algorithms, data structures, and system
primitives
in the "{domain}" domain of computing.

For each pattern, output in this EXACT format (one per entry, separated
by ---):

PATTERN: {snake_case_name}
INPUTS: {type1}, {type2}, ...
OUTPUT: {type}
KEYWORDS: {kw1}, {kw2}, {kw3}, ...
DESCRIPTION: {one line description}
---

Type system:
- Primitive: Int, Float, Bool, String
- Generic: List[T], Tuple[T1, T2], Option[T]
- Wildcard: Any

```

The prompt is intentionally rigid. LLMs follow structured output formats reliably when the format is unambiguous and the constraints are explicit. The `---` separator enables tolerant block-level parsing: if one pattern block is malformed, the parser skips it without corrupting subsequent entries.

3.2 System Message

The system message primes the LLM for exhaustive enumeration:

“You are a computational pattern database enumerator. Output patterns in the exact structured format requested. Be exhaustive. Cover the entire domain.”

The emphasis on exhaustiveness is deliberate. LLMs tend toward brevity; without this directive, they produce 10–20 patterns per domain instead of 50–100.

3.3 Configuration

Parameter	Default	Purpose
<code>model</code>	<code>claude-sonnet-4-5-20250929</code>	Enumeration model
<code>max_tokens</code>	8192	Per-call token budget
<code>temperature</code>	0.3	Low temperature for consistent enumeration
<code>timeout_secs</code>	60	Call timeout
<code>patterns_per_domain</code>	100	Requested pattern count per domain

Temperature 0.3 balances enumeration completeness against output consistency. Higher temperatures produce more creative pattern names but also more malformed entries. Lower temperatures produce more consistent formatting but may omit less common patterns.

3.4 Async/Sync Bridge

The Cartographer creates a Tokio runtime per LLM call via `tokio::runtime::Runtime::new().block_on()`. This bridge pattern is safe because:

1. Enumeration is a batch operation, not a latency-sensitive runtime path
2. Each domain enumeration is independent — no shared state between calls
3. The same pattern is used by L3 hybrid synthesis, validated in production

4. Type Parsing

4.1 The Problem

LLMs do not produce perfectly consistent type annotations. A prompt requesting `List[Int]` may receive `List<Int>`, `Vec[Int]`, `list[int]`, or `[Int]`. The type parser must be tolerant of these variations while mapping everything into

the Pattern type system.

4.2 Type System

The Pattern type system comprises 8 types:

Type	Variants Accepted	Maps To
<code>Int</code>	<code>int</code> , <code>integer</code> , <code>i64</code> , <code>i32</code> , <code>usize</code>	<code>GraphType::Int</code>
<code>Float</code>	<code>float</code> , <code>f64</code> , <code>f32</code> , <code>double</code>	<code>GraphType::Float</code>
<code>Bool</code>	<code>bool</code> , <code>boolean</code>	<code>GraphType::Bool</code>
<code>String</code>	<code>string</code> , <code>str</code> , <code>text</code> , <code>&str</code>	<code>GraphType::String</code>
<code>List<T></code>	<code>List[T]</code> , <code>List<T></code> , <code>Vec[T]</code> , <code>Vec<T></code>	<code>GraphType::List(Box<T>)</code>
<code>Tuple<T...></code>	<code>Tuple[T1, T2, ...]</code>	<code>GraphType::Tuple(Vec<T>)</code>
<code>Option<T></code>	<code>Option[T]</code> , <code>Optional[T]</code>	<code>GraphType::Option(Box<T>)</code>
<code>Any</code>	<code>any</code> , <code>*</code> , <code>T</code> , <code>generic</code>	<code>GraphType::Any</code>

4.3 Parsing Strategy

The parser uses a three-level strategy:

1. **Exact match on normalized lowercase** — handles all primitive types and their aliases
2. **Prefix-based wrapper detection** — handles `List[T]`, `Vec[T]`, `Option[T]`, `Tuple[T1, T2]` with both square brackets and angle brackets
3. **Fallback to `Any`** — unknown types (e.g., `HashMap[String, Int]`) map to the polymorphic wildcard

Nested types are handled recursively: `List[List[Float]]` correctly produces `GraphType::List(Box::new(GraphType::List(Box::new(GraphType::Float)))`.

4.4 Bracket Normalization

The `strip_wrapper` function accepts both Python-style brackets (`List[Int]`) and generic-style brackets (`List<Int>`). This dual acceptance is essential because different LLMs — and even the same LLM across different prompts — use both notations interchangeably.

4.5 Top-Level Splitting

The `split_top_level` function splits comma-separated type arguments while respecting nesting depth. The depth counter tracks `[`, `<`, `(` as open delimiters and `]`, `>`, `)` as close delimiters. Only commas at depth 0 produce splits. This handles `Tuple[List[Int], Option[String]]` correctly.

5. The DynamicCatalog

5.1 Structure

The `DynamicCatalog` is a runtime-extensible pattern registry with three index structures:

```
pub struct DynamicCatalog {  
  patterns: Vec<DiscoveredPattern>,    // Primary storage  
  by_name: HashMap<String, usize>,     // Exact name lookup → O(1)  
  by_keyword: HashMap<String, Vec<usize>>, // Keyword index → O(k)  
    lookup  
  by_domain: HashMap<String, Vec<usize>>, // Domain index → O(1) per  
    domain  
}
```

All three indexes are maintained incrementally on `add()`. No rebuild step is required.

5.2 DiscoveredPattern

Each pattern record carries full provenance:

Field	Type	Purpose
name	String	Snake_case canonical name
inputs	Vec<GraphType>	Typed input signature
output	GraphType	Typed output
category	String	Domain category
keywords	Vec<String>	Searchable keywords
description	String	One-line description (used as L3 scaffold)
domain	String	Source domain enumeration
source_model	String	LLM that enumerated this pattern
discovered_at	u64	Unix timestamp (milliseconds)

5.3 Keyword Search

The DynamicCatalog implements Jaccard similarity search over keywords:

$$J(Q, P) = \frac{|Q \cap P|}{|Q \cup P|}$$

Where Q is the query keyword set and P is the pattern's keyword set. A configurable threshold filters low-similarity matches.

The search is index-accelerated: the `by_keyword` inverted index identifies candidate patterns that share at least one keyword with the query. Only candidates are scored, avoiding a full scan of the catalog.

Example: - Pattern `rate_limiter` with keywords `{"rate", "limit", "throttle"}` - Query `{"rate", "limit"}` - Intersection: `{"rate", "limit"}` (2 elements) - Union: `{"rate", "limit", "throttle"}` (3 elements) - Jaccard similarity: $2/3 = 0.67$

Results are returned sorted by descending similarity score.

5.4 Persistence

The `DynamicCatalog` uses bincode serialization [5] — the same binary format used by `LoMemory` and `StyleMemory` throughout the Code Factory. The `persistence_path` field is marked `#[serde(skip)]` to avoid embedding absolute paths in serialized data.

Two persistence modes:

1. **Explicit save:** `catalog.save()` serializes to the configured path
2. **Load-or-create:** `DynamicCatalog::with_persistence(path)` loads from disk if the file exists, otherwise creates an empty catalog with the path configured for future saves

A typical catalog of 100 patterns serializes to approximately 15 KB in binary format.

5.5 PatternSig Conversion

The `to_pattern_sigs()` method converts dynamic patterns into the `PatternSig` format used by composition tables, graph validation, and the OODA engagement loop. Names are prefixed with `dyn:` to maintain namespace separation:

```
token_bucket → dyn:token_bucket  
round_robin_scheduler → dyn:round_robin_scheduler
```

The `dyn:` prefix is the boundary marker between the validated static catalog (800 patterns, 100% benchmark coverage) and the dynamically discovered catalog (LLM-enumerated, lazily validated on first use).

6. Deduplication

6.1 Two-Level Deduplication

The Cartographer deduplicates discovered patterns against two sources:

1. **Static catalog** — the 800 patterns in the base Code Factory, stored as a `HashSet<String>` of canonical names at initialization
2. **Dynamic catalog** — previously discovered patterns already in the `DynamicCatalog`, checked via `lookup_by_name()`

A pattern is considered a duplicate if its normalized `snake_case` name matches any existing entry in either catalog. This name-based deduplication is intentionally conservative: two patterns with different names but identical semantics (e.g., `selection_sort` and `straight_selection_sort`) will both be admitted. The cost of a redundant entry is negligible; the cost of filtering a legitimate pattern is loss of coverage.

6.2 Name Normalization

Pattern names are normalized during parsing:

- Spaces are replaced with underscores
- The entire name is lowercased
- Empty names are rejected
- Patterns with no inputs are rejected (every computational pattern transforms something)

7. Integration

7.1 CodeFactory Integration

The `CodeFactory` — the primary interface to the Code Factory system — accepts a `DynamicCatalog` via `set_dynamic_catalog()`. Once set, dynamic patterns participate in the full resolution chain:

```
Mount (L0 cache) → SideControl (Enlisted, static 800) → Doctrine (NCO recipes)
  → Guard (composition) → HalfGuard (decomposition) → Scramble (adaptive retry)
    → Tap (L2 pure synthesis → L3 hybrid LLM)
```

Dynamic patterns are accessible at the `SideControl` position (if their code has been cached by L0), through composition with static patterns (`Guard` and `Doctrine` positions), and as L3 scaffolds (`Tap` position, where the pattern's

description serves as a synthesis prompt).

7.2 Graph Validation

Pattern programs (typed DAGs) validate pattern references against the combined static + dynamic catalog. A graph node referencing `dyn:round_robin_scheduler` passes validation if and only if a dynamic pattern with that name exists in the DynamicCatalog and its type signature is compatible with the graph's edge types.

```
let mut combined: Vec<PatternSig> = build_pattern_signatures(); // static
                                800
combined.extend(dynamic_catalog.to_pattern_sigs());           //
                                dynamic N
let errors = graph.validate(&combined);                       // 0
                                errors
```

7.3 Lazy Code Generation

The Cartographer discovers signatures only — no implementations. Code generation follows a lazy evaluation strategy:

1. **First invocation:** No code exists. The OODA engagement loop reaches the Tap position. L2 pure synthesis attempts deterministic generation (deep composition, template synthesis, CEGIS). If L2 fails, L3 hybrid synthesis generates code via scaffold-augmented LLM prompting, using the pattern's description and type signature as the scaffold.
2. **Verification:** Generated code passes deterministic syntax checking and confidence scoring (+0.10 for scaffold match, +0.10 for keyword presence, +0.10 for structural correctness).
3. **Permanent caching:** Lo learned memory absorbs the verified result. Subsequent invocations resolve in microseconds — the LLM is never consulted again for this pattern.

This lazy strategy means the Cartographer can enumerate thousands of patterns at signature-only cost, and the system pays the code generation cost only for patterns that are actually used. Patterns that are enumerated but never invoked incur zero generation overhead.

7.4 Composition Expansion

Each dynamic pattern automatically expands the composition space. A dynamic catalog of N patterns adds up to $N \times (800 + N)$ potential composition pairs to the existing 227,653 type-compatible pairs from the static catalog. Type compatibility filtering applies identically: `SynthType::compatible_with()` checks structural compatibility, eliminating approximately 45% of candidate pairs.

8. Reporting

The Cartographer produces a structured report after enumeration:

Field	Type	Description
<code>domains_enumerated</code>	<code>usize</code>	Number of domains successfully processed
<code>patterns_discovered</code>	<code>usize</code>	Total patterns in the catalog after enumeration
<code>patterns_added</code>	<code>usize</code>	New patterns added in this run
<code>by_domain</code>	<code>HashMap<String, usize></code>	Patterns added per domain
<code>elapsed_secs</code>	<code>f64</code>	Total wall-clock time
<code>errors</code>	<code>Vec<(String, String)></code>	Domain-level errors (timeouts, provider failures)

The report enables monitoring of catalog growth and identification of domains where enumeration was incomplete due to provider errors or timeouts.

9. Evaluation

9.1 Parsing Robustness

The type parser handles the full range of LLM output variations:

Input	Parsed Result	Strategy
<code>Int</code>	<code>GraphType::Int</code>	Exact match
<code>i64</code>	<code>GraphType::Int</code>	Alias match
<code>integer</code>	<code>GraphType::Int</code>	Alias match
<code>List[Int]</code>	<code>GraphType::List(Int)</code>	Square bracket wrapper
<code>List<Int></code>	<code>GraphType::List(Int)</code>	Angle bracket wrapper
<code>Vec[String]</code>	<code>GraphType::List(String)</code>	Vec alias
<code>List[List[Float]]</code>	<code>GraphType::List(List(Float))</code>	Recursive nesting
<code>Tuple[Int, Float, Bool]</code>	<code>GraphType::Tuple([Int, Float, Bool])</code>	Top-level split
<code>Option[String]</code>	<code>GraphType::Option(String)</code>	Option wrapper
<code>Optional[Int]</code>	<code>GraphType::Option(Int)</code>	Optional alias
<code>HashMap[String, Int]</code>	<code>GraphType::Any</code>	Unknown → fallback

In our testing with pre-canned LLM responses simulating the `os_kernel` domain, the parser successfully extracted 5/5 patterns with correct types from a representative response containing `List[Tuple[Int, Int]]`, `List[List[Int]]`, `Option[Int]`, and primitive types.

9.2 Deduplication Effectiveness

Deduplication operates at two levels:

- **Against static catalog:** Given a static catalog containing `gcd` and `fibonacci`, patterns with those names are filtered from LLM output. In

test: 3 discovered → 1 retained (2 duplicates correctly filtered).

- **Against dynamic catalog:** Given a dynamic catalog already containing `token_bucket`, a second discovery of `token_bucket` is filtered. In test: 2 discovered → 1 retained (1 duplicate correctly filtered).

9.3 Keyword Search Quality

Jaccard similarity search produces ranked results that correctly prioritize specific matches over partial overlaps:

Query	Match	Jaccard Score
<code>["rate", "limit"]</code>	<code>rate_limiter</code> (keywords: rate, limit, throttle)	0.67
<code>["rate", "limit"]</code>	<code>rate_counter</code> (keywords: rate, counter, metric)	0.25
<code>["scheduler", "round", "robin"]</code>	<code>round_robin_scheduler</code> (keywords: scheduler, round, robin, process, time)	0.50
<code>["memory", "allocator"]</code>	<code>buddy_allocator</code> (keywords: buddy, allocator, memory, block)	0.40

Threshold selection controls precision/recall tradeoff: threshold 0.2 admits broad matches (`rate_counter` at 0.25), while threshold 0.3 restricts to stronger keyword overlap.

9.4 Persistence

Binary serialization round-trips without data loss. In test: a 3-pattern catalog serializes, deserializes, and retains all entries, indexes, and metadata. File-based persistence loads correctly on restart.

9.5 Test Suite

The Cartographer module passes 14 unit tests covering:

Test	Assertion
<code>test_parse_type_primitives</code>	All 9 primitive type aliases parse correctly
<code>test_parse_type_list</code>	Nested lists and Vec alias
<code>test_parse_type_tuple</code>	Multi-element tuples with nested types
<code>test_parse_type_option</code>	Option and Optional alias
<code>test_parse_type_unknown_fallback</code>	Unknown types → Any
<code>test_parse_type_angle_brackets</code>	<code><></code> bracket syntax
<code>test_parse_enumeration_basic</code>	3 patterns with full type checking
<code>test_parse_enumeration_tolerant</code>	Skips malformed blocks, retains valid
<code>test_dedup_against_static</code>	Filters static catalog duplicates
<code>test_dedup_against_dynamic</code>	Filters dynamic catalog duplicates
<code>test_catalog_persistence_roundtrip</code>	Serialize/deserialize fidelity
<code>test_keyword_lookup_jaccard</code>	Ranked results with threshold filtering
<code>test_to_pattern_sigs</code>	dyn: prefix and type conversion
<code>test_all_domains_count</code>	48 domains enumerated

10. Related Work

10.1 LLM-Based Code Generation

Systems like GitHub Copilot [6], Cursor, and Claude Code use LLMs to generate code on every invocation — each request triggers a new generation pass. The Cartographer inverts this: the LLM enumerates patterns once, the system retrieves forever. This is analogous to the distinction between interpreted and compiled languages: Copilot interprets; the Cartographer compiles.

10.2 Program Synthesis

Traditional program synthesis systems [7] generate programs from specifications (input/output examples, logical constraints). The Cartographer does not synthesize — it catalogs. Synthesis (via L2 and L3) occurs downstream, informed by the catalog’s type signatures and descriptions.

10.3 API Discovery

Automated API discovery tools [8] mine documentation and code repositories to extract function signatures. The Cartographer mines LLM knowledge directly, bypassing the need for existing documentation. This is particularly valuable for system-level domains (`os_kernel`, `device_driver`) where no single codebase contains all relevant patterns.

10.4 Knowledge Distillation

The Cartographer can be viewed as a form of structured knowledge distillation [9] — extracting specific, actionable knowledge from a large model into a compact, searchable format. Unlike traditional distillation (which produces a smaller model), the Cartographer produces a database: indexed, typed, and queryable in constant time.

11. Future Work

11.1 Multi-Model Consensus

Running enumeration against multiple LLMs (Claude, GPT-4, Llama, Qwen) and intersecting results would increase confidence in the completeness and correctness of discovered type signatures. Patterns confirmed by 3+ models receive higher trust scores.

11.2 Incremental Enumeration

Current enumeration processes entire domains in single calls. Incremental enumeration would present the existing catalog to the LLM and ask “what’s missing?” — reducing redundant discovery and focusing LLM effort on genuinely unknown patterns.

11.3 Type Signature Refinement

Some patterns have polymorphic type signatures that `Any` captures too loosely. A refinement pass could use concrete examples to tighten type constraints: `any_function : (Any) -> Any` might refine to `hash_map_get : (String, List[Tuple[String, Any]]) -> Option[Any]` through example-driven type inference.

11.4 Cross-Domain Composition Discovery

The Cartographer currently enumerates patterns within domains independently. Cross-domain enumeration — asking the LLM to identify patterns that compose across domain boundaries (e.g., a filesystem pattern that uses a compression pattern) — would directly expand the composition table.

11.5 Self-Termination

The catalog expansion process should have a well-defined termination criterion. When successive enumeration passes discover fewer than k new patterns per domain, the Cartographer has reached coverage saturation. At this point, the LLM’s contribution to the catalog is complete — the deterministic system has absorbed everything the LLM knows.

References

-
- [1] D. J. Charlot, “The Pattern Programming Language: Programs as Typed Pattern Graphs,” *AGI-Wisdom Technical Report*, February 2026.

- [2] D. J. Charlot, “Cortex: The Universal AI Compiler,” *AGI-Wisdom Technical Report*, January 2026.
- [3] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021. (HumanEval benchmark)
- [4] J. Austin et al., “Program Synthesis with Large Language Models,” *arXiv preprint arXiv:2108.07732*, 2021. (MBPP benchmark)
- [5] D. Tolnay, “bincode — A compact binary serialization format,” *crates.io*, 2024.
- [6] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *OpenAI Codex*, 2021.
- [7] S. Gulwani, O. Polozov, and R. Singh, “Program Synthesis,” *Foundations and Trends in Programming Languages*, vol. 4, no. 1-2, pp. 1-119, 2017.
- [8] R. Pandita et al., “Inferring Method Specifications from Natural Language API Descriptions,” *ICSE*, 2012.
- [9] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *arXiv preprint arXiv:1503.02531*, 2015.